Group 07 Progress Report: Log-Based Anomaly Detection on HDFS: A Supervised vs. Unsupervised Comparison

Aditya Tripuraneni, Vanessa Lai, Harmanpreet Singh Sagar
{tripuraa,laiv3,sagarh}@mcmaster.ca

1 Introduction

Distributed systems record their activity in logs, and engineers rely on these logs to find out when something has gone wrong. The volume of logs makes manual inspection impractical: the HDFS cluster in our dataset produced over 11 million log lines in under two days, roughly 290,000 lines per hour (Zhu et al., 2023; He et al., 2016). Failures are also rare, affecting only about 3% of sessions, so they are easy to miss in the data. Supervised models can learn to recognize these failures, but they need labeled examples, and labeling logs at this scale takes time and money. This raises the question of whether labels are necessary at all. Our main research question is whether unsupervised models, which train without needing labels, can detect anomalies as well as supervised models. We will compare two supervised classifiers (logistic regression and XGBoost) with two unsupervised detectors (Isolation Forest and a DeepLog-style LSTM) using the same preprocessing and the same train/validation/test split.

2 Related Work

Log-based anomaly detection has been approached from several directions. Early methods represent each session by its event counts. Xu et al. (2009) apply PCA to message-count vectors from console logs, and He et al. (2016) compare supervised logistic regression, decision trees, and SVMs with unsupervised PCA, invariant mining, and log clustering on HDFS and BGL logs. These count-vector methods are the starting point for our logistic regression, XGBoost, and Isolation Forest models.

Deep sequence models instead learn what should happen next. DeepLog (Du et al., 2017) trains an LSTM to predict the next event from the preceding ones and flags events that violate its predictions, which is the idea behind our LSTM. LogAnomaly (Meng et al., 2019) builds on this idea with an unsupervised LSTM that also uses event counts and semantic template vectors, and reports an average F1 of 0.95 on HDFS.

Comparisons between these families depend on the evaluation protocol. Chen et al. (2022) evaluate five neural networks used by six methods (four unsupervised, two supervised) and conclude that "[s]upervised methods generally achieve superior performance than unsupervised methods do." Le and Zhang (2022) warn that randomly shuffling sequences "may lead to the data leakage problem in the training phase," and state that because the HDFS dataset "does not contain timestamp information," a chronological split cannot be applied to it. Landauer et al. (2024) further show that HDFS is highly repetitive: its 575,061 sequences reduce to "26,814 (4.7% of total sequences) unique sequences," which comprise "only 666 (2.5% of unique sequences or 0.1% of total sequences) unique count vectors." They also find that simple detectors "achieve competitive detection rates compared to advanced state of the art approaches," indicating that "high detection rates are easy to achieve in some data sets." Because the preprocessed HDFS sessions we use carry no timestamps, we also split sessions at random, so we read our results as a controlled comparison of four models on one split and not as absolute scores comparable with published ones.

3 Dataset

3.1 Source and Collection

We use the HDFS_v1 dataset from Loghub (Zhu et al., 2023; Jiang et al., 2024). Its logs were generated by running benchmark workloads on a 203-node Hadoop Distributed File System (HDFS) cluster (Zhu et al., 2023; Xu et al., 2009) and contain 11,175,629 log lines, about 1.5 GB (He et al., 2016). HDFS splits files into large blocks, replicates each block on multiple DataNodes (three

Sessions (block IDs)	575,061
Anomalous sessions	16,838 (2.93%)
Event types	29
Length: median / 99th pct / max	19 / 33 / 298

Table 1: Summary of the HDFS_v1 sessions. Counts agree with He et al. (2016); length statistics are our own computation.

by default), and records block locations on a NameNode (Shvachko et al., 2010). In the data, every log line contains a block ID, and most logged block sizes are 64 MB (94% are 67,108,864 bytes). Grouping the lines by block ID yields one *session* per block (Zhu et al., 2023; He et al., 2016).

3.2 Properties of the Data

One data point is one block session: the time-ordered sequence of log events for a single block, with a binary normal/anomalous label. Loghub defines 29 event templates (E1–E29), message types in which variable parts such as block IDs and IP addresses are replaced by wildcards (Zhu et al., 2023; Jiang et al., 2024). All 29 occur in the sessions, matching the 29 message types reported by Xu et al. (2009). Reading the template names, a typical normal session follows a regular lifecycle: a block is allocated (E22), received by three DataNodes (E5, E11, E9), recorded by the NameNode (E26) once per replica, then deleted (E23, E21), so the same events repeat three times, once per replica (Shvachko et al., 2010). Anomalous sessions break this pattern. For example, `[E5, E5, E22, E7, E11, E10, E14, E7]` contains three exception events (E7, E10, E14) and never reaches E26.

Table 1 summarizes the data. It is highly imbalanced: 16,838 of the 575,061 sessions are anomalous (He et al., 2016), i.e., 2.93%, and we reproduced these counts from the Loghub files. Sessions are short (median 19 events), and length does not separate the classes (median 19 for normal, 20 for anomalous). We found no missing values and no duplicate block IDs.

3.3 Preprocessing

Our proposal planned to parse the raw logs into event templates with Drain (He et al., 2017). We found that Loghub already provides parsed event templates and per-block event sequences, so we use those directly. The remaining conversion is handled by our own loader functions, which keeps the pipeline simple. The loader (1) joins each block's event sequence with its label by block ID, checking the join is exactly one-to-one; (2) converts each stored sequence string into a list of integer event IDs; (3) drops the `Type`, `Latency`, and `TimeInterval` columns, where `Type` holds the anomaly type that Loghub provides alongside the labels (Zhu et al., 2023) and is populated for exactly the anomalous sessions, so using it would leak the label, and the other two fall outside our model input (the event sequence alone); and (4) encodes labels as 0 (normal) and 1 (anomalous).

3.4 Train, Validation, and Test Splits

Our dataset has a 70/10/20 train/validation/test split. We stratified the dataset on the binary label so each split keeps the same 2.93% anomaly rate. Without stratification, a random split could put too few of the rare anomalies into validation or test, and the four models would no longer be compared on the same class balance.

3.5 Annotation

We did not annotate any data. The labels were assigned by the dataset's authors, who manually labeled each block session as normal or anomalous using handcrafted rules (Zhu et al., 2023). We treat them as ground truth.

4 Features

Each block session is represented in one of two ways, depending on the model. In both cases, the model only uses the sequence of event IDs from the session (Section 3).

Event-count vectors. Logistic regression, XGBoost, and Isolation Forest use a count-based representation. For each session, we create a vector with one entry for every event type, where each value records how many times that event appears in the session. This is a common representation for HDFS session classification (He et al., 2016). The event vocabulary is built using only the training split, resulting in 29 features. If an event type appears only in the validation or test set, it is ignored so that information from those splits does not leak into training.

Some events can occur many times in the same session, especially when a block is read repeatedly, which makes the raw counts highly skewed. To reduce this effect, we apply $\log(1+x)$ to each count before training the models. The main drawback of this representation is that it only captures how often events occur and completely ignores their order.

This is one reason we also use an LSTM-based model.

Ordered event sequences. The DeepLog-style LSTM (Du et al., 2017) uses the events in their original order. Each event type seen in the training data is assigned an integer token, along with an additional token for events that were never seen during training. The model is trained to predict the next event in a session. For each event, the previous 10 events are used as the input window and the current event is used as the target. At the beginning of a session, shorter windows are left-padded with a padding token.

We also add an end-of-session token to every sequence so that the model can learn when a normal session is expected to stop. This is important for unusually short sessions such as `[E22, E5]`, where the events themselves may look normal even though the session ends too early. Each token is mapped to a 16-dimensional embedding that is learned together with the LSTM, allowing the model to learn its own representation of the event types rather than relying on manually designed features.

Selection and augmentation. We do not perform feature selection or data augmentation. Since there are only 29 event types, all count-based features are kept. For logistic regression, the L2 regularization term helps limit the influence of features that are not useful for classification. As additional representations for the count-based models, we plan to test TF-IDF weighting and event n-gram counts. These may provide more information about which events are important while also preserving some local event ordering.

5 Implementation

Logistic regression. Our supervised baseline is the scikit-learn implementation of logistic regression, applied to the 29-dimensional event-count vectors (Section 4). Each count x is first replaced by $\log(1+x)$, so that a few very large counts, such as a block that is read hundreds of times, do not dominate the others. The model outputs the probability that a session is anomalous. It is trained by minimizing the class-weighted log loss with an L2 penalty, $\frac{1}{2}\|w\|^2 + C\sum_i s_i\,\ell_i$, where ℓ_i is the log loss of session i and s_i is the weight of its class. We use scikit-learn's "balanced" weights, which are inversely proportional to class frequency: an anomalous training session has weight 17.1 and a normal one 0.52, about 33 times less. The objective is optimized with L-BFGS, which converged in 33 iterations. The regularization strength C and the decision threshold are chosen on the validation set (Section 6).

Isolation Forest. Isolation Forest (Liu et al., 2008) is an unsupervised method that flags sessions that are unusual compared with the rest of the data. It builds random trees, each of which repeatedly splits the sessions on a random feature and a random value. Unusual sessions are isolated after only a few splits, while typical sessions need many, so a short average path length across the trees indicates an anomaly. We use the scikit-learn implementation with 100 trees, trained on the $\log(1+x)$ event-count vectors of the training split. Labels are used only afterwards, to choose a decision threshold on the validation split and to evaluate the model.

DeepLog-style LSTM. The LSTM (input described in Section 4) has two stacked layers with hidden size 64 and is trained to predict each next event, including the end-of-session token. The loss is cross-entropy over the next event, optimized with Adam (learning rate 10^{-3}, batch size 1024) for 10 epochs on 50,000 training sessions, during which the training loss fell from 0.545 to 0.253.

Issues encountered. Most of our difficulties come from the properties of the dataset. First, HDFS is extremely repetitive. Landauer et al. (2024) report only 666 unique count vectors among its 575,061 sessions, and our copy has 589. In our split, 99.94% of test sessions have a count vector that also appears in the training split, including 98.46% of the anomalous ones (only 52 of 3,368 anomalous test sessions have a pattern not seen in training). Count-based supervised models can therefore do very well largely by recognizing patterns they have already seen, which helps explain the high logistic regression scores. We read our results as a comparison between models and not as a guarantee for new logs. Second, anomalies are rare (2.93%), which led us to use class weights, F1 thresholds tuned on validation, and PR-AUC instead of accuracy. Third, the unsupervised models never see labels by design, so they cannot filter out the roughly 3% of anomalous sessions in their training data. Finally, the LSTM is our most expensive model, so we trained it on a random subset of 50,000 sessions (272 seconds). It saw only 28 of

the 29 event types, and an event type it has never seen is treated as highly surprising.

6 Results and Evaluation

Protocol. We use the same stratified 70/10/20 split with a fixed random seed for all models (Section 3). This gives us 402,541 training, 57,507 validation, and 115,013 test sessions. All results so far are based on this single split, which allows us to compare models on exactly the same test data. The validation set also contains 1,684 anomalous sessions, giving us enough examples to tune the models. We plan to add 5-fold cross-validation to see how much the results change across different splits (Section 7).

We use the validation set to choose hyperparameters and decision thresholds, then evaluate each model once on the test set. For the unsupervised models, labels are not used during training and are only used to select the decision threshold on validation.

Metrics. Since only 2.93% of sessions are anomalous, accuracy alone would be misleading. A model that predicts every session as normal would already achieve about 97.1% accuracy without detecting a single anomaly. Instead, we focus on precision, recall, and F1 for the anomalous class. We also report the area under the precision–recall curve (PR-AUC), calculated using average precision. This measures how well a model ranks anomalous sessions above normal ones across different thresholds and is more useful than ROC-AUC when the classes are highly imbalanced (Saito and Rehmsmeier, 2015).

Each model produces a continuous anomaly score. We choose the threshold that gives the highest F1 on the validation set and use that same threshold for testing. For logistic regression, we also tested regularization strengths C from $\{10^{-2}, \ldots, 10^{3}\}$ and selected $C = 100$ based on validation PR-AUC. For the LSTM, we calculate a session's anomaly score using the largest $-\log p$ assigned to any of its actual next events. This means an event that the model considers highly unlikely can cause the session to be flagged as anomalous, similar to the probability cutoff used in DeepLog.

Baseline. As a simple baseline, we use a majority-class predictor that labels every session as normal. Since it never predicts an anomaly, both its recall and F1 are zero. Its PR-AUC is also equal to the proportion of anomalous sessions in the dataset, which is 0.029.

Model	P	R	F1	PR-AUC
Majority class	0.000	0.000	0.000	0.029
Supervised				
Logistic Regression	0.991	0.995	0.993	0.997
Unsupervised				
Isolation Forest	0.944	0.567	0.709	0.714
DeepLog LSTM	0.476	0.661	0.553	0.472
top-k rule (k=5)	0.478	0.131	0.205	0.107

Table 2: Test-set precision (P), recall (R), F1, and PR-AUC on the anomalous class. Thresholds were chosen on validation. XGBoost is still in progress.

Results. Table 2 summarizes our results so far. All models perform better than the majority-class baseline. We also found that the validation and test F1 and PR-AUC scores differ by less than 0.01 for each of the three models (the top-k variant differs by about 0.02 in F1), suggesting that the thresholds selected during validation work well on unseen data.

Logistic regression performs the best by a large margin, reaching an F1 of 0.993 and a PR-AUC of 0.997. This suggests that, when labels are available, simply knowing which events occur and how often they appear is enough to distinguish almost all normal and anomalous sessions. The model does not need to consider event order to achieve strong results on this dataset.

Among the unsupervised models, Isolation Forest achieves an F1 of 0.709. Its precision is high at 0.944, meaning that most sessions it flags are actually anomalous. However, its recall is only 0.567, so it misses almost half of the anomalies. One possible explanation is that some anomalous sessions have event counts similar to normal sessions, making them difficult to separate using count-based features alone.

The DeepLog LSTM has a higher recall of 0.661, meaning it detects more anomalies than Isolation Forest, but its precision drops to 0.476 because it also flags many normal sessions. Looking more closely at its mistakes, we found that many missed anomalies are short, incomplete sessions such as `[E22, E5]`. These patterns appear repeatedly in the dataset, and since the unsupervised models are trained without removing the roughly 3% of sessions that are anomalous (Isolation Forest on all 402,541 training sessions, the LSTM on a random subset of 50,000), the LSTM can learn to treat them

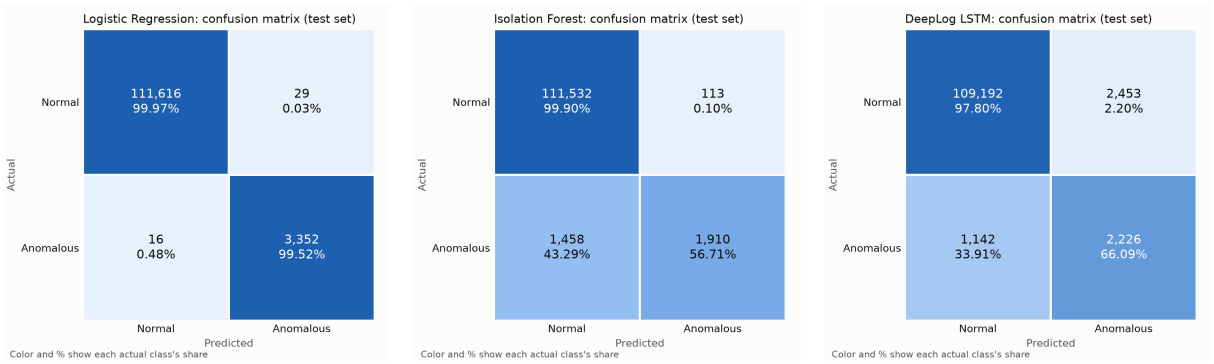


Figure 1: Test-set confusion matrices (115,013 sessions) for logistic regression, Isolation Forest, and the DeepLog LSTM, at the thresholds chosen on validation. Each cell shows the number of sessions and its share of the actual class (row), so the rare anomalous row stays readable next to the large normal row.

as normal. On the other hand, many of its false positives are longer normal sessions where events from different replicas appear in an unusual order.

We also compared our probability-based anomaly score with the original DeepLog top-k approach, which flags a session when the actual next event is not among the model's top k predictions. With $k = 5$, this approach performs much worse, reaching an F1 of only 0.205 compared with 0.553 for our probability-based score. We found that some rare error events still appear among the top few predictions, even when their predicted probabilities are low. The top-k method therefore misses them, while the probability-based score can still identify them as unusual.

Adding the end-of-session token also made a noticeable difference. When we removed it, the LSTM's validation F1 dropped from 0.561 to 0.500, and its PR-AUC fell from 0.469 to 0.353. This suggests that learning when a session should end helps the model detect incomplete sequences that might otherwise look normal.

Figure 1 shows how each model's errors are split. Of the 3,368 anomalous test sessions, logistic regression misses only 16 and raises 29 false alarms among 111,645 normal sessions. Isolation Forest raises just 113 false alarms but misses 1,458 anomalies, whereas the LSTM misses fewer anomalies (1,142) at the cost of 2,453 false alarms. The two unsupervised models therefore fail in opposite ways: Isolation Forest is conservative, while the LSTM trades many false alarms for extra recall.

Overall, our results so far show a clear advantage for supervised learning on the HDFS dataset. Logistic regression performs particularly well using only event counts, while the unsupervised models struggle with anomalies that resemble normal sessions. Even the strongest unsupervised model, Isolation Forest, falls behind logistic regression by about 0.28 F1. This suggests that having labelled examples makes a substantial difference for anomaly detection in this dataset.

7 Feedback and Plans

We plan to use 5-fold cross-validation over the training and validation data, keeping the test set fixed, to measure how much each model's scores and chosen thresholds vary across folds.

7.1 TA Feedback

We received feedback from our TA, Hongyi, who suggested we include significance tests for our results. We plan to use McNemar's test to compare each unsupervised model against each supervised model. McNemar's test fits our setup because every model makes a binary prediction on the same test sessions, and it only looks at the sessions where the two models disagree. Because McNemar's test only tells us whether two models differ, we will also report 95% confidence intervals for F1 and PR-AUC to show how large the difference is.

7.2 Reflections and Next Steps

Our results so far show that the supervised baseline is hard to beat and that the unsupervised models, especially the LSTM, leave room for improvement. Our first priorities are to implement XGBoost, so that both families have two models on the same split, and to run the label-fraction experiment from our proposal, retraining the supervised models on 1%, 10%, and 100% of the labels to measure how much the labels are worth. For the LSTM, we trained on only 50,000 sessions that still contain anomalies, so we plan to train it on more of the

training split and tune its window and hidden size on validation. We also plan to try TF-IDF and n-gram features for the count-based models (Section 4). Because HDFS has so few unique count vectors, we will also report how each model does on test sessions whose pattern was not seen in training (only 52 anomalous test sessions), so the results do not rest only on repeated patterns. Finally, we will add the 5-fold cross-validation, McNemar's tests, and confidence intervals described above, so that the differences between models are supported by more than a single split.

Team Contributions

Aditya Tripuraneni set up the repository and the shared pipeline (data loading and splits, count-vector features, evaluation metrics, and plotting), wrote the dataset download script and dataset primer, and implemented and evaluated the logistic regression baseline with its tests. He wrote the Dataset section of this report (except the splits subsection) and set up its citations. XGBoost is still in progress.

Vanessa Lai implemented and evaluated the Isolation Forest detector, including its script, tests, metrics, and confusion matrix. She set up the LaTeX report and wrote the Introduction, the splits subsection, the Isolation Forest paragraph of Implementation, the TA Feedback subsection, and the future plans.

Harmanpreet Singh Sagar implemented and evaluated the DeepLog-style LSTM on ordered event sequences, including its script, tests, metrics, and confusion matrix, and added the sequence-feature code. He wrote the Features and Results and Evaluation sections of this report.

References

Zhuangbin Chen, Jinyang Liu, Wenwei Gu, Yuxin Su, Jieming Zhu, Yongqiang Yang, and Michael R. Lyu. 2022. Experience report: Deep learning-based system log analysis for anomaly detection. *Preprint*, arXiv:2107.05908.

Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1285–1298.

Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)*, pages 33–40.

Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. 2016. Experience report: System log analysis for anomaly detection. In *Proc. IEEE 27th Int. Symp. Software Reliability Engineering (ISSRE)*, pages 207–218.

Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R. Lyu. 2024. A large-scale evaluation for log parsing techniques: How far are we? In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 223–234.

Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques. *Proc. ACM Softw. Eng.*, 1(FSE):1354–1375.

Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: How far are we? In *Proc. 44th Int. Conf. Software Engineering (ICSE)*, pages 1356–1367.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422.

Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, pages 4739–4745.

Takaya Saito and Marc Rehmsmeier. 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432.

Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. 2010. The Hadoop distributed file system. In *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, MSST '10, pages 1–10, USA. IEEE Computer Society.

Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. 2009. Detecting large-scale system problems by mining console logs. In *Proc. 22nd ACM Symp. Operating Systems Principles (SOSP)*, pages 117–132.

Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A large collection of system log datasets for AI-driven log analytics. In *IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 355–366.